

— Databases

Hoofdstuk 2: Databankontwerp

L. Espeel



— 4 fasen ontwikkelen informatiesysteem

1. Analyse

- Consulting: gesprekken voeren met de klant. Klant zegt zijn of haar wensen. De consultant vertaalt de wensen naar degene die het uiteindelijk zal moeten gaan maken.

2. Logisch ontwerp

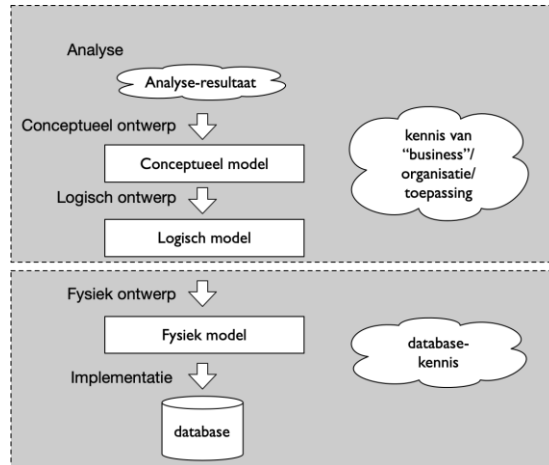
- Ontwerp relationele databank: relationele databank wordt ontworpen, gebaseerd op huidige behoeften en rekening houdend met toekomstige behoeften.
We doen dit via een **Entity Relationship Model (ERM)** en dit wordt getoond met een **Entity Relationship Diagram (ERD)** en is een uitstekende basis voor een fysiek databankontwerp.

3. Fysiek ontwerp

- Mapping: omzetten van het logische model naar een fysiek model dat zich richt op 1 van de belangrijkste DBMS-typen: relationeel, hiërarchisch, netwerk of object-georiënteerd. Het bevat de structuur van de tabellen, alle kenmerken van de gegevens en richt zich op de details van de implementatie.

4. Bouw

Bouwen van een databank



3

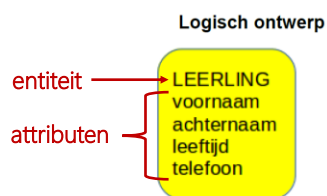
Logisch ontwerp

4

— Logisch ontwerp

Een logisch ontwerp bestaat uit entiteiten en attributen.

- **Entiteit:** het object waarvan informatie wordt opgeslagen, is steeds in het **enkelvoud**
- **Attribuut:** een eigenschap
- **Waarden**



Uiteindelijke tabel

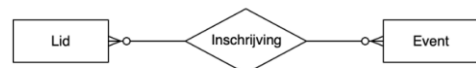
Leerlingen			
voornaam	achternaam	leeftijd	telefoonnr
Piet	Konijn	13	0611211211
Jan	Haas	15	
Sylvia	Slak	14	0600700700

kolom = attribuut = veld

rij = record =
instantie van
entiteit

5

— Logisch model ⁽¹⁾



Entity-Relationship diagram - eerste stap

Lid = een entiteit

Event = een entiteit

Inschrijving = een relatie tussen *Lid* en *Event*

- **Entiteiten** zijn dingen die betekenis hebben in de organisatie. Een entiteit is in een Entity Relationship Model (ERD) altijd een **zelfstandig naamwoord** in een **rechthoek**.
- **Relatie** geeft het verband tussen twee of meer entiteiten weer. De relatie wordt altijd weergegeven met een **werkwoord** (inschrijven) of een **werkwoordsvorm** (inschrijving) en staat in een **ruit**.

6

Logisch model (2)



Entity-Relationship diagram - eerste stap

Multipliciteit of kardinaliteit van de relatie kan je zien aan de hand van een ERD. Multipliciteit of kardinaliteit kunnen we in het geval van databanken vereenvoudigen tot een maat voor het aantal elementen in een verzameling.

In het ERD hierboven zien we dus:

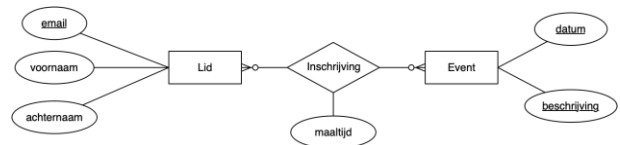
- Elk *Lid* heeft zich ingeschreven voor 0 of meer *Events*
- Elk *Event* heeft 0 of meer ingeschreven *Leden*

De relatiesymbolen volgen de kraaienpootnotatie, die standaard wordt gehanteerd in MySQL Workbench.

elke A heeft 0 of 1 B's	
elke A heeft precies 1 B	
elke A heeft 0 of meer B's	
elke A heeft 1 of meer B's	

7

Logisch model (3)



In de bovenstaande figuur zien we het volgende:

- de entiteit *Lid* heeft als attributen: email-adres, *voornaam*, *achternaam*
- de entiteit *Event* heeft als attributen: *datum*, *beschrijving*
- de relatie *Inschrijving* heeft als attribuut: *maaltijd(keuze)*

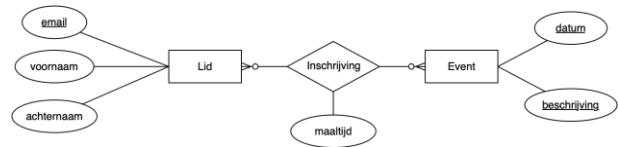
De attributen per entiteit en relatie worden soms opgesplitst en in cirkels geplaatst.

Je ziet dat bepaalde attributen onderstreept zijn. Dit wil zeggen dat deze attributen een entiteit identificeren. Het onderlijnde attribuut is een unieke identificatie (UID) van de entiteiten.

- Een *email* identificeert een *Lid*
- Combinatie van *datum* en *beschrijving* identificeert een *Event*

8

— Logisch model (4)



Sommige attributen zijn niet geschikt om te gebruiken in een databank omdat ze **vluchtig** zijn. Een voorbeeld is het veld *leeftijd*.

- Dit zou moeten bijgewerkt worden na iedere verjaardag. Vervang het attribuut beter door een niet-vluchtige eigenschap zoals *geboortedatum*.

Sommige velden moeten verplicht ingevuld zijn in een record. Dit zijn **verplichte attributen**. Andere zijn **optionele attributen**.

Het is belangrijk in een databank dat iedere record uniek is. Geen enkele record mag exact dezelfde zijn. Om dit te garanderen maakt men gebruik van **unieke identifiers (UID)**. Dit kan 1 attribuut of een combinatie van meerdere attributen zijn. In het fysieke model wordt de UID de **primaire sleutel** of **primary key**.

9

— Logisch model (5)

Notaties:

= primaire UID

(#) = secundaire UID: reserve en controlemiddel

* = verplicht

o = optioneel

LEERLING
leerlingnummer
(#) voornaam
(#) achternaam
(#) geboortedatum
* adres
* woonplaats
* postcode
o telefoon

10

— Relaties (1)

Zoals hier eerder al werd besproken kunnen entiteiten onderling een relatie hebben. Relaties:

- Tonen iets belangrijks
- Geven aan hoe entiteiten zich tot elkaar verhouden
- Komen in **paren**: van A naar B en van B naar A
- Hebben een mate van **kardinaliteit** (= hoeveelheid)

11

— Relaties (2)

De relaties schrijven we op een bepaalde manier zodat ze later gemakkelijk in een ERD kunnen getekend worden, dit word **ERDish** genoemd. Voorbeeld:

- Iedere klant koopt één of meer producten.
- Ieder product wordt gekocht door nul of meer klanten.

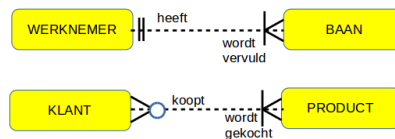
Toelichting:

- Elke zin begint met *iedere* of *elke*
- De zinnen komen in paren
- Er zit een werkwoord in de zin
- 1 of meer, precies 1 en 0 of meer geeft de kardinaliteit aan

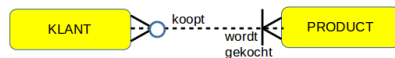
12

— Relaties (3)

- Elke werknemer heeft 1 of meer banen.
- Iedere baan wordt door precies 1 werknemer vervuld.



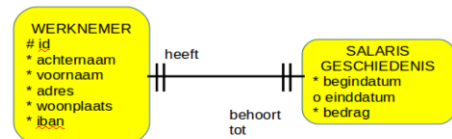
- Iedere klant koopt 1 of meer producten.
- Ieder product wordt gekocht door nul of meer klanten.



13

— Relaties (4)

— Hierbij erft de kind entiteit de UID van de ouder entiteit; hieronder staat een voorbeeld.



- Elke werknemer heeft exact 1 salarisgeschiedenis.
- Elke salarisgeschiedenis behoort tot exact 1 werknemer.

De doorgetrokken lijn maakt duidelijk dat de UID van de werknemer wordt overgedragen op de UID van salarisgeschiedenis, dit is een **sterke relatie**.

In het fysieke model (zie straks) komt hiervoor een extra kolom *werknemer_id* in de tabel *salarisgeschiedenis*

14

— ERD versus ERM

Een **Entity Relationship Diagram (ERD)** geeft dus de entiteiten die van belang zijn weer en de relaties die tussen die entiteiten bestaan. Het doel van een ERD is:

- Een voorstel te documenteren waarvoor discussie kan plaatsvinden.
- Te kijken of er geen redundantie is (informatie mag slechts 1 keer getoond worden in een ERD).

Een **Entity Relationship Model (ERM)** is een logisch model voor relationele databanken.

- Bevatten meestal een ERD.
- Bevatten ook informatie die niet in de ERD zijn opgenomen: **datatypen** en **constraints** (beperkingen)
 - **Constraints** zijn beperkingen of voorwaarden die voortvloeien uit bedrijfsregels. Het informatiesysteem moet bijvoorbeeld een seintje geven als iemand 25 jaar in dienst is. Dit staat niet in het ERD maar wel in het ERM.

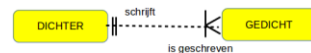
15

— Overdraagbaarheid

Sommige relaties zijn niet **overdraagbaar** of **transferable**. Dit moet in het ERD gedocumenteerd worden.

Voorbeeld 1:

- Een dichter schrijft 1 of meerdere gedichten.
- Een gedicht is geschreven door exact 1 dichter.



Een gedicht dat eenmaal is geschreven door een bepaalde dichter kan later niet door een andere dichter zijn geschreven. De relatie tussen gedicht en dichter is dus niet overdraagbaar.

Voorbeeld 2:

- Een leerling zit in exact 1 klas.
- Een klas bestaat uit 1 of meerdere leerlingen.



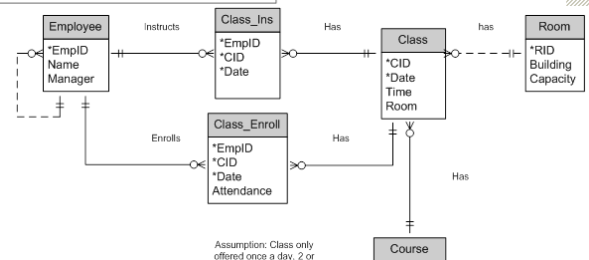
De relaties hier zijn beiden overdraagbaar want een leerling kan achteraf naar een andere klas worden verplaatst en een klas kan uit nieuwe leerlingen komen te bestaan.

16

— CRUD analyse

Een ander onderdeel van het ERM is de **CRUD** analyse. Er moet genoteerd worden welke data gemaakt, opgehaald, aangepast en verwijderd moet kunnen worden.

Create	Welke data wordt aangemaakt?
Retrieve	Welke data wordt opgehaald?
Update/Read	Welke data wordt ververs?
Delete	Welke data wordt gewist?



17

— Normalisatie

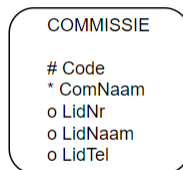
Bij het bepalen van de juiste entiteiten en attributen kan normaliseren een belangrijk hulpmiddel zijn. Er zijn verschillende niveaus mogelijk, in de praktijk gaan we vaak t.e.m. 3NF:

- UNF: Unnormalized form
- 1NF: First normal form
- 2NF: Second normal form
- 3NF: Third normal form
- EKNF: Elementary key normal form
- BCNF: Boyce-Codd normal form
- 4NF: Fourth normal form
- ETNF: Essential tuple normal form
- 5NF: Fifth normal form
- DKNF: Domain-key normal form
- 6NF: Sixth normal form

18

— Normalisatie – niet-normaalvorm (UNF)

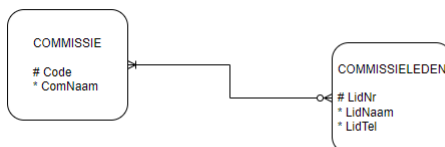
In een niet-genormaliseerd basismodel worden de entiteit en de attributen netjes genoteerd.



19

— Normalisatie – eerste normaalvorm (1NF) ⁽¹⁾

Een entiteit is in de eerste normaalvorm als de attributen niet meer dan 1 waarde kunnen hebben en de attributen zelf ook maar 1 keer voorkomen.



20

— Normalisatie – eerste normaalvorm (1NF) ⁽²⁾

Onderstaande lijkt OK te zijn, maar ... wat als een klant 2 telefoonnummers heeft?



Zo oplossen?

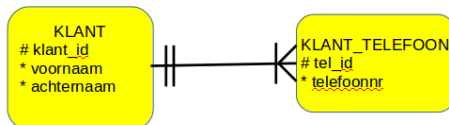


Maar ... wat als het nog meer dan 2 telefoonnummers heeft?

21

— Normalisatie – eerste normaalvorm (1NF) ⁽³⁾

Betere oplossing:

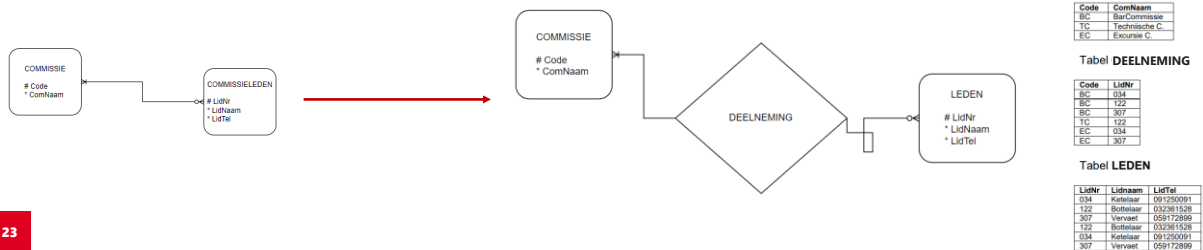


22

— Normalisatie – tweede normaalvorm (2NF)

Gegevens die niet helemaal functioneel afhankelijk zijn van de primaire sleutel leiden tot redundantie wat vroeg of laat kan leiden tot inconsistentie (niet-overeenstemming van gegevens).

We lossen dit op door de gegevens die niet van de primaire sleutel afhankelijk zijn op te splitsen in een aparte tabel.

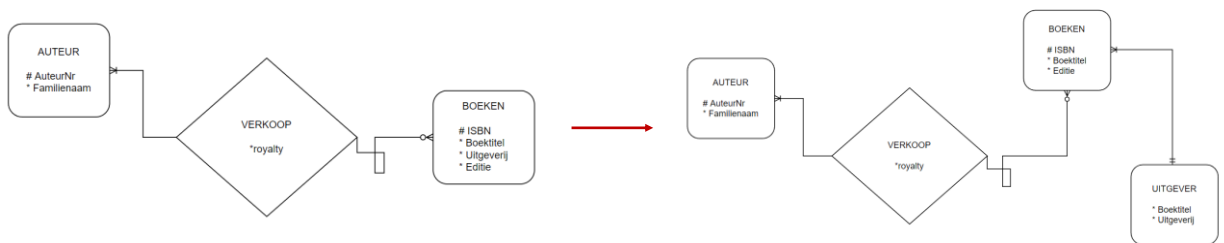


23

— Normalisatie – derde normaalvorm (3NF)

Bij de derde normaalvorm gaan we nog verder op zoek naar gegevens die functioneel afhankelijk zijn van én de primaire sleutel én van een ander niet-sleutel gegeven veld, zijn transitief afhankelijk en leiden ook tot gegevensovertolligheid.

In ons voorbeeld van commissieleden hebben we dit niet, dus starten we met een nieuw voorbeeld. Ook hier moet je alle transitieve afhankelijkheden afsplitsen in nieuwe tabellen.

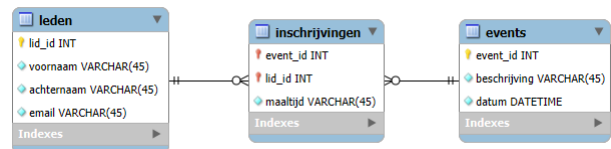


24

Fysiek ontwerp

25

Fysiek model (1)



Afhankelijk van het type DBMS dat voor de databank gebruikt wordt, bouwen we het fysieke model voor de databank.

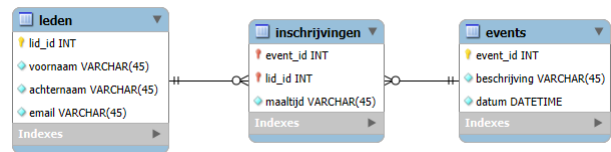
- Een fysiek model voor een NoSQL-databank zal er anders uitzien dan het fysieke model voor een relationele databank. Wij focussen ons in deze cursus vooral op relationele databanken.

Vertaling van het logisch model naar een fysiek model:

- Entiteiten worden in deze fase **tabellen**: *Leden* en *Events*.
- De relatie *Inschrijving* wordt een **tabel**: *Inschrijvingen*
- De attributen worden **velden** (kolommen) in deze tabellen.
- Voor elk attribuut is een **datatype** bepaald (hier een *INT* en *VARCHAR(30)*).
- Alle attributen voor 1 rij vormen samen een **record**.
- De UID voor elke record wordt de **primaire sleutel** of **primary key**.

26

— Fysiek model (2)

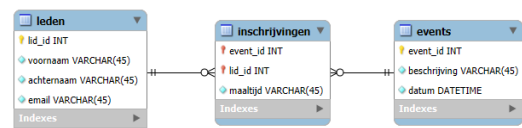


Als we naar de relaties kijken tussen de tabellen in het fysieke model kunnen we het volgende opmerken:

- Per record in de tabel *Inschrijvingen* hebben we een verwijzing naar exact 1 record uit de tabel *Leden*.
- Per record uit de tabel *Leden* kunnen er 0, 1 of meerdere verwijzingen naar records in de tabel *Inschrijvingen* zitten.
- Per record in de tabel *Inschrijvingen* hebben we een verwijzing naar exact 1 record uit de tabel *Events*.
- Per record uit de tabel *Events* kunnen er 0, 1 of meerdere verwijzingen naar records in de tabel *Inschrijvingen* zitten.

27

— Logisch versus fysiek model



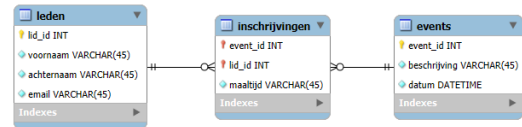
Logisch model	Fysiek model
Elk <i>Event</i> bevat 0 of meerdere ingeschreven <i>Leden</i> .	• Per record in de tabel <i>Inschrijvingen</i> hebben we een verwijzing naar exact 1 record uit de tabel <i>Leden</i>. • Een record uit de tabel <i>Leden</i> kan 0, 1 of meerdere verwijzingen in records in de tabel <i>Inschrijvingen</i> hebben.
Elk <i>Lid</i> schrijft zich in voor 0 of meerdere <i>Events</i> .	• Per record in de tabel <i>Inschrijvingen</i> hebben we een verwijzing naar exact 1 record uit de tabel <i>Events</i>. • Een record uit de tabel <i>Events</i> kan 0, 1 of meerdere verwijzingen in records in de tabel <i>Inschrijvingen</i> hebben.

28

— Fysiek model

In een fysiek model valt het volgende op:

- **Tabelnamen** staan nu in het **meervoud**.
- **Veldnamen** starten met een **kleine letter** en worden afgekort om de tabellen overzichtelijk te houden.
- Aan elke veldnaam wordt een **datatype** gekoppeld. Een datatype is het soort data dat wordt opgeslagen.
- Ook hier wordt in MySQL Workbench standaard de kraaienpootnotatie gehanteerd. Soms vereenvoudigt men dit echter tot **één-op-veel symbolen** (symbolen 1 en ∞), dit is ook instelbaar.

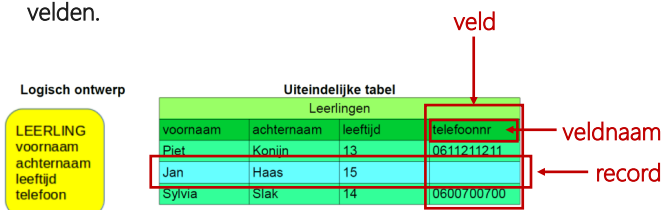


29

hogeschool
vives

— Bouw van een relationele databank (1)

- Een relationele databank bestaat uit een verzameling **tabellen** van gelijkvormige gegevens.
- Een tabel is een verzameling van **records**.
- Een record is een verzameling van **veldwaarden**.
 - Als er **geen waarde** is, spreken we van **null**. Een 0 of een lege stringwaarde (afhankelijk van het datatype) is echter wel een waarde.
- Wanneer je de databank gaat bouwen, worden de **entiteiten** de **tabellen** en de **attributen** de **velden**.



30

hogeschool
vives

Bouw van een relationele databank (2)

Relationele databank: wordt zo genoemd omdat een **tabel** eigenlijk een relatie voorstelt. Het is een relatie tussen verschillende velden die samen een tabel vormen. Je kunt dan ook gaan rekenen met die tabellen met relationele operaties.

- selectie in een tabel A op basis van de voorwaarde *geslacht* = "v"

selectie
(van rijen)

lidnr	voornaam	achternaam	geslacht
2913	Hanna	de Vries	v
3362	Marie	van Doorn	v

- projectie van een tabel A op basis van een reeks veldnamen: *lidnr, voornaam, geslacht* from *leden*

projectie
(van kolommen)

lidnr	voornaam	geslacht
1027	Jan	m
2913	Hanna	v
2934	Klaas	m
3362	Marie	v

- Je kan een selectie en een projectie ook gaan combineren:

lidnr	voornaam	geslacht
2913	Hanna	v
3362	Marie	v

combinatie van selectie en projectie

31

hogeschool
vives

Bouw van een relationele databank (3)

Probleem

In de onderstaande tabel zie je een probleem. Er is sprake van **redundantie**. Bepaalde elementaire waarden worden voor elke aanmelding herhaald.

Deze tabel is daardoor redundant ("overtollig") en dat moet je proberen vermijden want:

- Je hebt meer opslagruimte nodig dan strikt noodzakelijk
- Het is lastiger om de consistentie van de gegevens te garanderen.
 - Gegevens moeten altijd in alle versies up-to-date zijn.

lidnr	voornaam	achternaam	geslacht	event-datum	event-maaltijd
1027	Jan	Blokstra	m	12-06-2019	maaltijd-A
1027	Jan	Blokstra	m	19-06-2019	maaltijd-A
2913	Hanna	de Vries	v	12-06-2019	maaltijd-B
2913	Hanna	de Vries	v	12-06-2019	maaltijd-B
2934	Klaas	Bergstra	m	19-06-2019	maaltijd-A
3362	Marie	van Doorn	v	12-06-2019	maaltijd-B

32

hogeschool
vives

Bouw van een relationele databank (4)

Oplossing

Rechts werd de tabel van de vorige slide **genormaliseerd** (een vorm zonder redundantie). De redundantie werd verwijderd door een extra tabel *inschrijvingen* in te voeren. Elke inschrijving heeft als primaire sleutel een uniek *inschrijfnr.*

Natuurlijk hebben we ook een verwijzing van de ene tabel naar de andere nodig.

- Elke record in de tabel *inschrijvingen* heeft een **primaire sleutel** (*inschrijfnr*) en elke record in de tabel *leden* heeft een **primaire sleutel** (*lidnr*).
- Elke record in *inschrijvingen* verwijst via de **externe sleutel** of **foreign key** (*lidnr*) naar de juiste record in de tabel *leden*.
 - Een externe sleutel is dus een verwijzing van de ene tabel naar de andere. De primaire sleutel in de ene tabel is de externe sleutel in de verwijzende tabel.

leden

lidnr	voornaam	achternaam	geslacht
1027	Jan	Blokstra	m
2913	Hanna	de Vries	v
2934	Klaas	Bergstra	m
3362	Marie	van Doorn	v

Foreign key

inschrijvingen		event-datum	event-maaltijd
inschrijfnr	lidnr		
135	1027	12-06-2019	maaltijd-A
136	1027	19-06-2019	maaltijd-A
147	2913	12-06-2019	maaltijd-B
149	2913	12-06-2019	maaltijd-B
171	2934	19-06-2019	maaltijd-A
173	3362	12-06-2019	maaltijd-B

33

Bouw van een tabel (1)

Belangrijke regels:

- Elke record beschrijft een verschillende entiteit.
- Elke record moet uniek zijn

NOK!

voornaam	familienaam	geboortedatum	positie
Amber	Anjoux	01-04-2000	Middenvelder
Kimberley	Lunettes	28-12-2000	Doel
An-Sofie	Bigfeet	01-09-2000	Spits
Amber	Anjoux	01-04-2000	Middenvelder

34

— Bouw van een tabel (2)

Oplossing: **primaire sleutel** toevoegen, dit kan op één veld of een combinatie van velden.



lidnummer	voornaam	familienaam	geboortedatum	positie
2007103	Amber	Anjoux	01-04-2000	Middenvelder
2009086	Kimberley	Lunettes	28-12-2000	Doel
2008042	An-Sofie	Bigfeet	01-09-2000	Spits

35

— Bouw van een tabel (3)

Primaire sleutel kan ook een uniek volgnummer zijn, de veldnaam kies je vrij, maar bevat best "id" in de benaming.

- Er is een mogelijkheid van instelling waarbij DBMS elk toegevoegd record automatisch een uniek nummer aan toekent in de primaire sleutel. Dit wordt **autonummering** genoemd.



lid_id	voornaam	familienaam	geboortedatum	positie
1	Amber	Anjoux	01-04-2000	Middenvelder
2	Kimberley	Lunettes	28-12-2000	Doel
3	An-Sofie	Bigfeet	01-09-2000	Spits

36

— Opsplitsen van een tabellen

lidnummer	voornaam	familienaam	ploeg	coach
2007103	Amber	Anjoux	A-ploeg	Yorka Joris
2009086	Kimberley	Lunettes	B-ploeg	Annick Dadels
2008042	An-Sofie	Bigfeet	A-ploeg	Yorka Joris

Tabel *leden*

lidnummer	voornaam	familienaam	ploeg_id
2007103	Amber	Anjoux	A-ploeg
2009086	Kimberley	Lunettes	B-ploeg
2008042	An-Sofie	Bigfeet	A-ploeg

Tabel *ploegen*

ploeg_id	coach
A-ploeg	Yorka Joris
B-ploeg	Annick Dadels

37

— Relaties tabellen

primaire sleutel

Tabel *leden*

lidnummer	voornaam	familienaam	ploeg_id
2007103	Amber	Anjoux	A-ploeg
2009086	Kimberley	Lunettes	B-ploeg
2008042	An-Sofie	Bigfeet	A-ploeg

externe sleutel

primaire sleutel

Tabel *ploegen*

ploeg_id	coach
A-ploeg	Yorka Joris
B-ploeg	Annick Dadels

∞

1

Nu eens gebruikmakend van de één-op-veel notatie:

- 1-kant: elke ploeg komt slechts één keer voor. Een lid hoort dus tot 1 ploeg.
- ∞-kant: een ploeg kan meerdere keren voorkomen. Een ploeg kan dus meerdere leden bevatten.

38

— Referentiële integriteit ⁽¹⁾

In de tabel aan de veel-kant kan je geen record toevoegen met een refererende sleutel die niet bestaat in de tabel aan de één-kant.

Tabel *leden*

<u>lidnummer</u>	voornaam	familienaam	<i>ploeg_id</i>
2007103	Amber	Anjoux	A-ploeg
2009086	Kimberley	Lunettes	B-ploeg
2008042	An-Sofie	Bigfeet	A-ploeg
2006079	Paola	Smellekens	C-ploeg

Tabel *ploegen*

<u>ploeg_id</u>	coach
A-ploeg	Yorka Joris
B-ploeg	Annick Dadels

∞

1



39

— Referentiële integriteit ⁽²⁾

Referentiële integriteit geldt uiteraard ook voor autonummering-velden.

Tabel *leden*

<u>lidnummer</u>	voornaam	familienaam	<i>ploeg_id</i>
2007103	Amber	Anjoux	1
2009086	Kimberley	Lunettes	2
2008042	An-Sofie	Bigfeet	1

Tabel *ploegen*

<u>ploeg_id</u>	beschrijving	coach
1	A-ploeg	Yorka Joris
2	B-ploeg	Annick Dadels

∞

1

40

Referentiële integriteit (3)

Data dient trapsgewijs toegevoegd te worden, dus eerst in tabel *ploegen*, vervolgens in tabel *leden*.

Tabel *leden*

<u>lidnummer</u>	voornaam	familienaam	<i>ploeg_id</i>
2007103	Amber	Anjoux	A-ploeg
2009086	Kimberley	Lunettes	B-ploeg
2008042	An-Sofie	Bigfeet	A-ploeg
2) 2006079	Paola	Smellekens	C-ploeg

Tabel *ploegen*

<u>ploeg_id</u>	coach
A-ploeg	Yorka Joris
B-ploeg	Annick Dadels
1) C-ploeg	An Moerkerke

